Chapter 1 - Python Basics

January 10, 2021

1 What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991. This course is a introduction to both fundamental programming concepts and the Python programming language also in Embedded Python (Micro Python). Python 3 is the most up-to-date version of the language with many improvements made to increase the efficiency and simplicity of the code that you write.

1.1 It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting
- database operation and managing databases
- embedded programming.

1.2 What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

1.3 Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-orientated way or a functional way.

1.4 Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial. However, Python 2, although not being updated with anything other than security updates, is still quite popular.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

1.5 Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

1.6 Python for IoT

- It is gaining significant traction in recent times in the IoT development
- Due to the support for embedded programming Python and the vast library support
- IoT involves Wireless Sensor Networks, data analytics, machine learning, big data which are focused on real time analytics and processes.
- The development of the IoT solution requires a lightweight and scalable programming language
- Python suitable for data intensive applications
- Huge & active community that provides support and libraries for Python
- Python is portable, expandable and embeddable which makes it system independent and is suitable for a variety of single board computers regardless of the architecture and operating system
- Python is the language of choice for the Raspberry Pi

---

2 Python syntax

Indentation refers to the spaces at the beginning of a code line.

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

[ ]:
```
if 5 > 2:
    print("Five is greater than two!")
```

Python will give you an error if you skip the indentation:

[ ]:
```
# Syntax Error:
if 5 > 2:
print("Five is greater than two!")
```

The number of spaces is up to you as a programmer, but it has to be at least one.

You have to use the same number of spaces in the same block of code, otherwise Python will give you an error:

[ ]:
```
# Syntax error
if 5 > 2:
 print("Five is greater than two!")
        print("Five is greater than two!")
```

2.1 Commnents

A comment is a piece of text within a program that is not executed. It can be used to provide additional information to aid in understanding the code. The # character is used to start a comment and it continues until the end of the line.

[3]:
```
# Comment on a single line

user = 'Eero' #This is a comment
print(user)
```

```
Eero
```

Comments can:

Provide context for why something is written the way it is:

```
    # This variable will be used to count the number of times anyone tweets the word persnicke
    persnickety_count = 0
```

Help other people reading the code understand it faster:

```
    # This code will calculate the likelihood that it will rain tomorrow
    complicated_rain_calculation_for_tomorrow()
```

Ignore a line of code and see how a program will run without it:

```
    # useful_value = old_sloppy_code()
    useful_value = new_clean_code()
```

2.1.1 Identifier

In addition to keywords, there are various other elements in a Python program. They are called identifiers. A program can have keywords, variables, functions, classes, modules, packages etc. An identifier should start with either an alphabet (lower or upper case) or underscore (_). More than one alpha-numeric characters or underscores may follow.

```
Keywords are predefined. They are in lowercase. They can't be used for any other purpose.
Conventionally, name of class begins with uppercase alphabet. Others start with lowercase alpha
Single underscore in the beginning of a variable name is used to indicate a private variable.
Two underscores in beginning indicate that the variable is strongly private.
Two leading and trailing underscores are used in language itself for special purpose. For exam
```

Here are some examples of valid identifiers:

Student score
aTotal
sum_age here is double underscore at the beginning and/or the end of varieable ___ init___ and ___ count TotalValue price1 cost_of_item

---

2.2 Statements

Any text terminated by Enter key (called newline character) is recognized as a statement by Python interpreter. Hence, each line in the input cell of notebook is a statement. Occasionally, a Python statement may spill over multiple lines. To do so, use backslash () as continuation character.

[6]:
```
greeting="Hello World. \
            Welcome to the exciting world \
            of Python programming"
print ("Output")
greeting
```

```
Output
```

[6]:
```
'Hello World.            Welcome to the exciting world            of Python
programming'
```

However, if number of elements in other data types such list, tuple or dictionary spread over many lines, there is no need to use backslash to show continuation.

[7]:
```
numbers=[10, 20, 30, 40,
                              50, 60, 70, 80,90, 100]
print("Output")
numbers
```

```
Output
```

[7]:
```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

On the other hand, to write more than one statements in single line, use semicolon (;) to separate them.

[9]:
```
a=10; b=20; c=a+b; print("Output"); print (c)
```

```
Output
30
```

2.3 Indents

Multiple statements are usually grouped together to define a function, class or loop. Python uses mechanism of uniformly indented statements to denote such a group or block. **Usually a colon (:) symbol marks the beginning of block.**

When you press Enter key after colon mark, next line starts after leaving a white space from left margin – something similar to pressing Tab key. Every subsequent statement in the block follows same indent level. To mark end of the block, press the backspace and reduce the indentation level back to earlier level.

Look at the code snippet in the input cell. The code itself may not be understood at this level but don't worry. Just see how indent level increases after colon symbol.

[ ]:
```
num=int(input("enter a number"))
l=len(num)
app=0
for i in range(l):
    f=int(num[i])
    if app==0:
            app=app+1
            if i==0 : s=f
    else:
            if app>0:
                    if s==f:
                            app=app+1
                    else:
                            print (s,app)
                            app=1
                            s=f
print ("Output")
print (s,app)
```

2.4 Print

Now what we're going to do is teach our computer to communicate. The gift of speech is valuable: a computer can answer many questions we have about "how" or "why" or "what" it is doing. In Python, the print() function is used to tell a computer to talk. The message to be printed should be surrounded by quotes.

[2]:
```
from platform import python_version
"""""Here we import pythopn version() function from platform.
without this the code will not work"""""

# from Eero to H2C students
print("Happy coding !") # Notes these "text inside quotes"

# But if you print variable using function call
print('Python', python_version())
```

```
Happy coding !
Python 3.7.6
```

2.4.1 Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

A variable is created the moment you first assign a value to it. In the below example, we store the message "Hello there...." in a variable called message_string. Variables can't have spaces or symbols or ä, ö or any scandies in their names other than an underscore (_). They can't begin with numbers but they can have numbers after the first letter (e.g., cool_variable_5 is OK).

In Python, we assign variables by using the equals sign (=) like this way.

```
message_string = "Hello there H2C students"
# Prints "Hello there...."
print(message_string)
```

It's no coincidence we call these creatures "variables". If the context of a program changes, we can update a variable but perform the same logical process on it.

```
message_string = "Hello there H2C students"
# Prints "Hello there...."
print(message_string)

message_string = "Terve teille H2C opiskelijat"
```

```
# Prints "Hello there...."
print(message_string)
```

2.5 Strings

Computer programmers refer to blocks of text as strings. In our last exercise, we created the string "Hello world!". In Python a string is either surrounded by double quotes ("Hello world") or single quotes ('Hello world'). It doesn't matter which kind you use, just be consistent.

2.6 Calculations

Computers absolutely excel at performing calculations. The "compute" in their name comes from their historical association with providing answers to mathematical questions. Python performs addition, subtraction, multiplication, and division with +, -, *, and /.

```
# Prints "500"
print(573 - 74 + 1)

# Prints "50"
print(25 * 2)

# Prints "2.0"
print(10 / 5)
```

Notice that when we perform division, the result has a decimal place. This is because Python converts all ints to floats before performing division. In older versions of Python (2.7 and earlier) this conversion did not happen, and integer division would always round down to the nearest integer.

Division can throw its own special error: ZeroDivisionError. Python will raise this error when attempting to divide by 0.

Mathematical operations in Python follow the standard mathematical order of operations. Link: https://en.wikipedia.org/wiki/Order_of_operations

So, if we have to calculate these kind of equation 25 * 68 + 13 / 28 and print out the result

Simply we can do it like that:

```
print ((25 * 68)+ (13 / 28))

or like that way

print (25 * 68 + 13 / 28)

Output should be: 1700.4642857142858
```

2.7 Exponents

Python can also perform exponentiation. In written math, you might see an exponent as a superscript number, but typing superscript numbers isn't always easy on modern keyboards. Since this operation is so related to multiplication, we use the notation **.

```
# 2 to the 10th power, or 1024
print(2 ** 10)

# 8 squared, or 64
print(8 ** 2)

# 9 * 9 * 9, 9 cubed, or 729
print(9 ** 3)

# We can even perform fractional exponents
# 4 to the half power, or 2
print(4 ** 0.5)
```

Here, we compute some simple exponents. We calculate 2 to the 10th power, 8 to the 2nd power, 9 to the 3rd power, and 4 to the 0.5th power.

2.8 Concatenation

The + operator doesn't just add two numbers, it can also "add" two strings! The process of combining two strings is called string concatenation. Performing string concatenation creates a brand new string comprised of the first string's contents followed by the second string's contents (without any added space in-between).

```
greeting_text = "Hey there!"
question_text = "How are you doing?"
full_text = greeting_text + question_text

# Prints "Hey there!How are you doing?"
print(full_text)
```

In this sample of code, we create two variables that hold strings and then concatenate them. But we notice that the result was missing a space between the two, let's add the space in-between using the same concatenation operator!

```
full_text = greeting_text + " " + question_text

# Prints "Hey there! How are you doing?"
print(full_text)
```

Now the code prints the message we expected.

If you want to concatenate a string with a number you will need to make the number a string first, using the str() function. If you're trying to print() a numeric variable you can use commas to pass it as a different argument rather than converting it to a string.

```
birthday_string = "I am "
age = 10
birthday_string_2 = " years old today!"

# Concatenating an integer with strings is possible if we turn the integer into a string first
full_birthday_string = birthday_string + str(age) + birthday_string_2

# Prints "I am 10 years old today!"
print(full_birthday_string)

# If we just want to print an integer
# we can pass a variable as an argument to
# print() regardless of whether
# it is a string.

# This also prints "I am 10 years old today!"
print(birthday_string, age, birthday_string_2)
```

Using str() we can convert variables that are not strings to strings and then concatenate them. But we don't need to convert a number to a string for it to be an argument to a print statement.

2.9 Multi-line Strings

Python strings are very flexible, but if we try to create a string that occupies multiple lines we find ourselves face-to-face with a SyntaxError. Python offers a solution: multi-line strings. By using three quote-marks (""" or ''') instead of one, we tell the program that the string doesn't end until the next triple-quote. This method is useful if the string being defined contains a lot of quotation marks and we want to be sure we don't close it prematurely.

```
kalevala = """
Yks oli vanha Väinämöinen,
toin oli nuori Joukahainen
ajettih hyö vastakkahe,
aisapa tarttui aisan piähän
rahkis rahkehe takertui,
vesi tippui vempeleestä
usva uuen aisan piästä
rasva kiehui rahkehesta.
Virkkipä nuori Joukahainen:
"Sinä herra, minä herra,
kumpi meistä tien pitääpi?"
Virkkipä vanha Väinämöinen:
"Mitellähpä miekkojamme
koitellahpa konstiamme,
kummall on miekka pitempi
jolla on kortta korkiampi,
sehän tässä tien pitääpi."
"""
```

In the above example, we assign a famous Kalevala's words to a variable. Even though the quote contains multiple linebreaks, the code works!

If a multi-line string isn't assigned a variable or used in an expression it is treated as a comment.

2.10 Review

In this lesson, we accomplished a lot of things! We instructed our computers to print messages, we stored these messages as variables, and we learned to update those messages depending on the part of the program we were in. We performed mathematical calculations and explored some of the mathematical expressions that Python offers us. We learned about errors and other valuable skills that will continue to serve us as we develop our programming skills.

Good job!

[ ]:
```
# Here is some examples how to use variables

x = 5
y = "John"
nimi = 'Eero'
a = b = 3
print(x)
print(y)
print(a)
print(b)
print(nimi)

#Variables do not need to be declared with any particular type and can even␣
 ↪change type after they have been set.

x = 4 # x is of type int

print(x)

x = "Sally" # x is now of type str

print(x)
```

[ ]:
```

```